Объектно-ориентированное программирование

6 семестр
Лекция 6.

Содержание лекции:

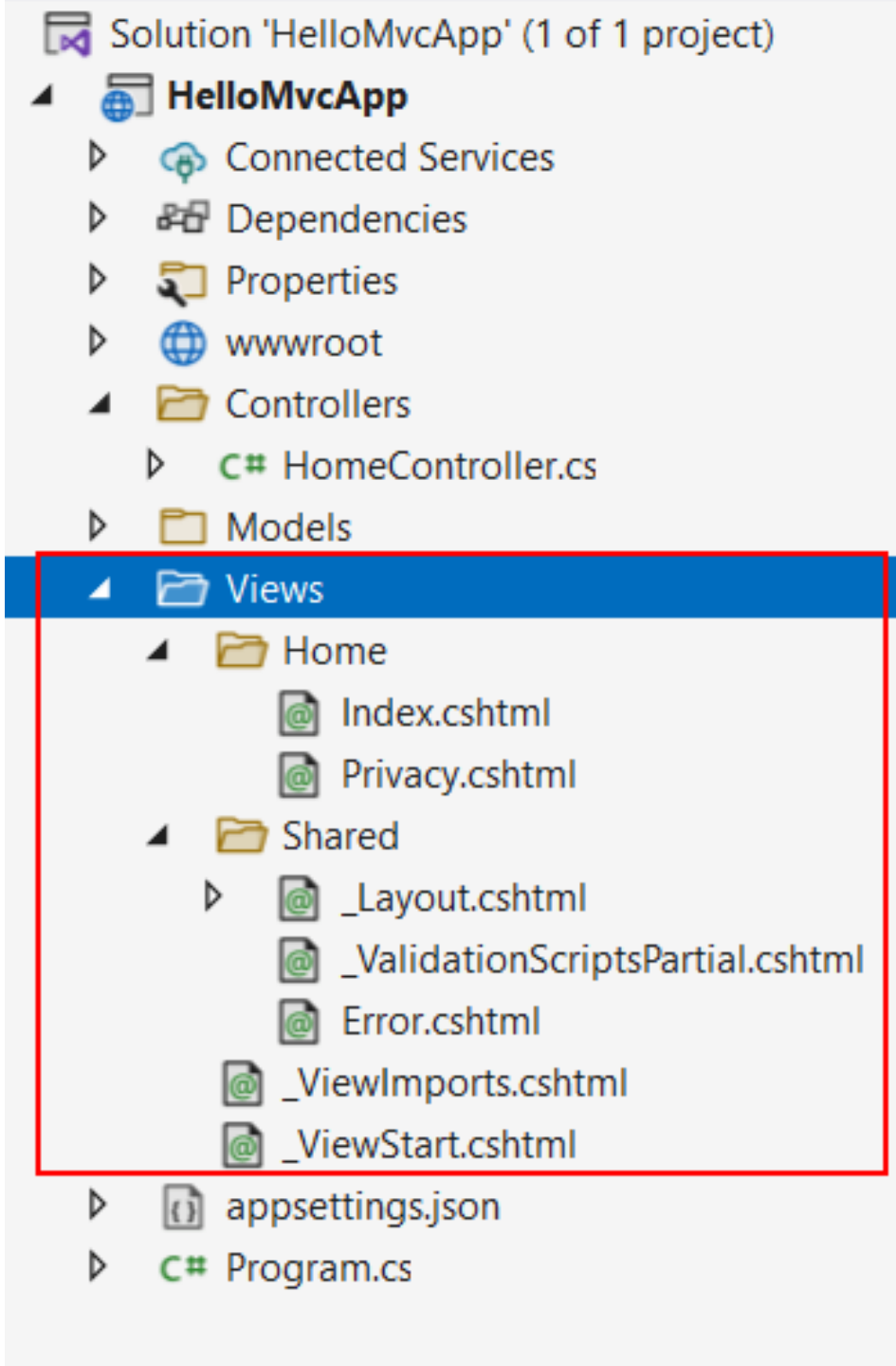
- Представления View
- движок представлений Razor
- Модели
- Tag-хелперы

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Представления

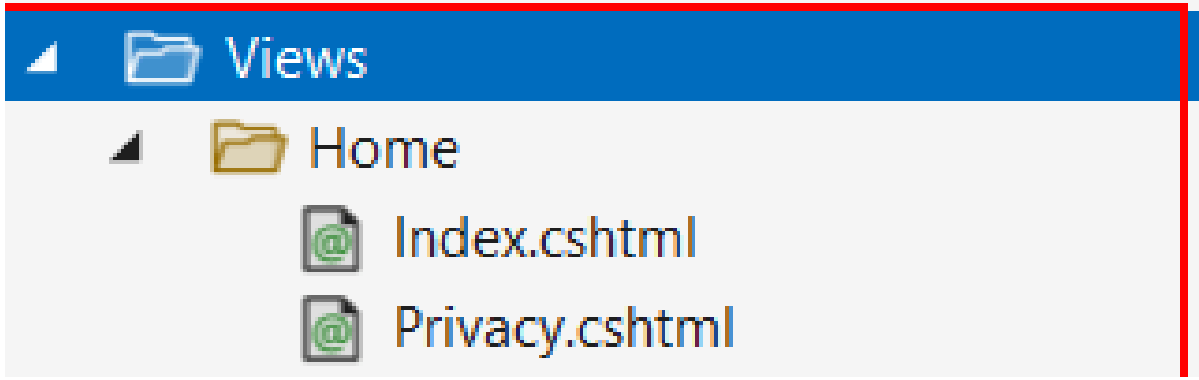
Обычно при обращении к веб-приложению пользователь ожидает получить веб-страницу с какими-нибудь данными. В MVC для этого, как правило, используются **представления**, которые определяют внешний вид приложения и на основе которых потом формируется веб-страница. В ASP.NET MVC Core представления - это файлы с расширением cshtml, которые содержат код пользовательского интерфейса в основном на языке html, а также конструкции Razor - специального движка представлений, который позволяет переходить от кода html к коду на языке C#.

Для хранения представлений в проекте ASP.NET Core предназначена папка **Views**. Например, если мы возьмем проект по типу **ASP.NET Core Web App (Model-View-Controller)**, то мы увидим, что он содержит ряд представлений:

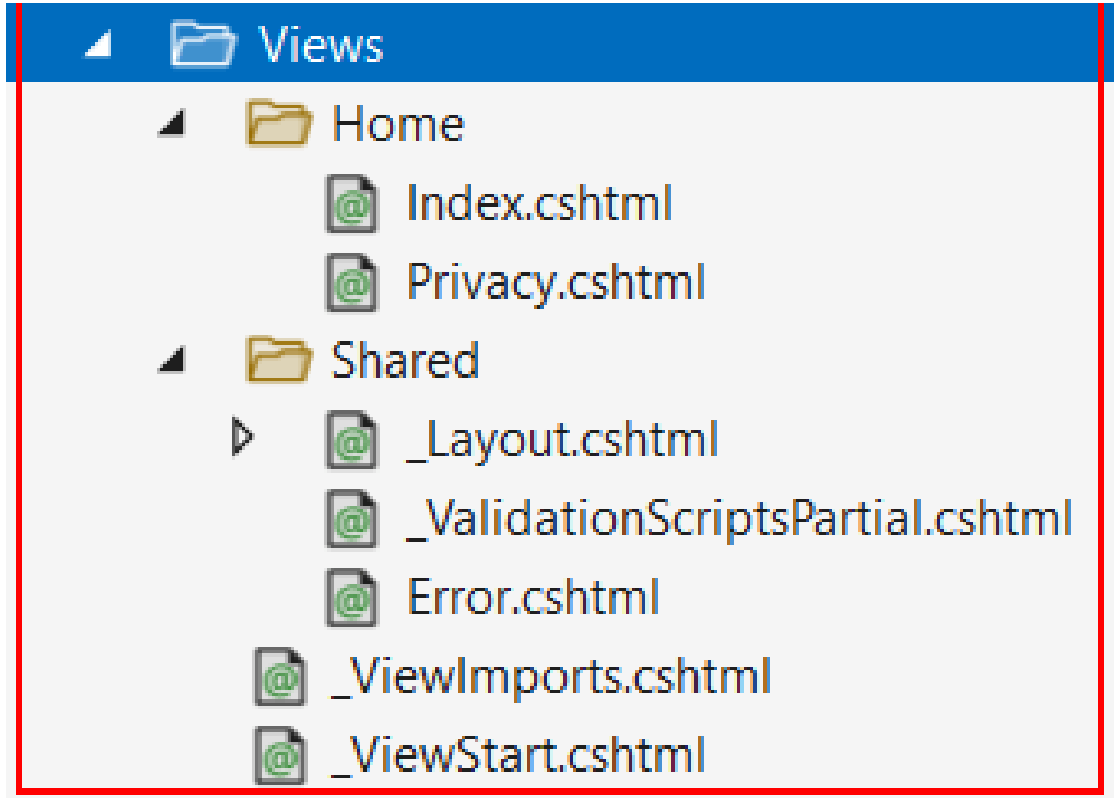


Подобный проект для хранения представлений в папке **Views** определяет некоторую структуру:

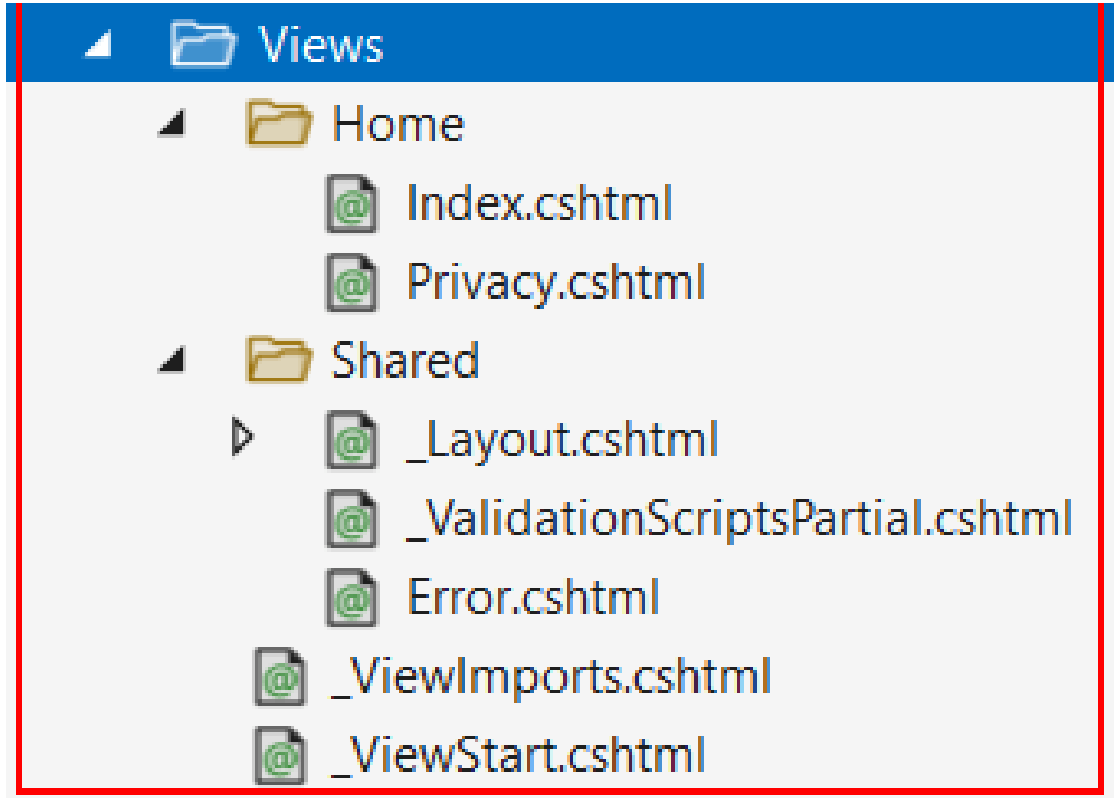
Во-первых, как правило, для каждого контроллера в проекте создается подкаталог в папке Views, который называется по имени контроллера и который хранит представления, используемые методами данного контроллера. Так, по умолчанию имеется контроллер HomeController и для него в папке Views есть подкаталог **Home** с представлениями для методов контроллера HomeController - в данном случае это файлы **Index.cshtml** и **Privacy.cshtml**.



Также здесь есть папка **Shared**, которая хранит общие представления для всех контроллеров. По умолчанию это файлы **_Layout.cshtml** (используется в качестве мастер-страницы), **Error.cshtml** (используется для отображения ошибок) и **_ValidationScripsPartial.cshtml** (частичное представление, которое подключает скрипты валидации формы).



И в корне каталога **Views** также можно найти два файла **_ViewImports.cshtml** и **_ViewStart.cshtml**. Эти файлы содержат код, который автоматически добавляется ко всем представлениям. *_ViewImports.cshtml* устанавливает некоторые общие для всех представлений пространства имен, а *_ViewStart.cshtml* устанавливает общую мастер-страницу.



Определим в проекте папку **Controllers**, в которую добавим новый контроллер - **HomeController** со следующим кодом:

```
public class HomeController : Controller
{
        public IActionResult Index()
        {
            return View();
        }
}
```

За работу с представлениями отвечает объект **ViewResult**. Он производит рендеринг представления в веб-страницу и возвращает ее в виде ответа клиенту. Чтобы возвратить объект ViewResult, в методе контроллера вызывается метод **View**:

Вызов метода **View** возвращает объект **ViewResult**. Затем уже **ViewResult** производит рендеринг определенного представления в ответ. По умолчанию контроллер производит поиск представления в проекте по следующим путям:

/Views/Имя_контроллера/Имя_представления.cshtml
/Views/Shared/Имя_представления.cshtml

Согласно настройкам по умолчанию, если название представления не указано явным образом, то в качестве представления будет использоваться то, имя которого совпадает с именем действия контроллера.

Создание представлений

Сначала создадим в проекте новую папку **Views**, а в ней определим новый каталог **Home** - для представлений контроллера HomeController.

Далее добавим в каталог **Views/Home** новый элемент по типу **Razor View - Empty**, который назовем **index.cshtml**:

Add New Item - MvcApp

Installed

Visual C#

General

ASP.NET Core

Online

Sort by: Default

Search (Ctrl+E)

API Controller - Empty	Visual C#
API Controller with read/w...	Visual C#
Razor Page - Empty	Visual C#
Razor View - Empty	Visual C#
Razor Layout	Visual C#
Assembly Information File	Visual C#
Code File	Visual C#
Machine Learning Model (...	Visual C#

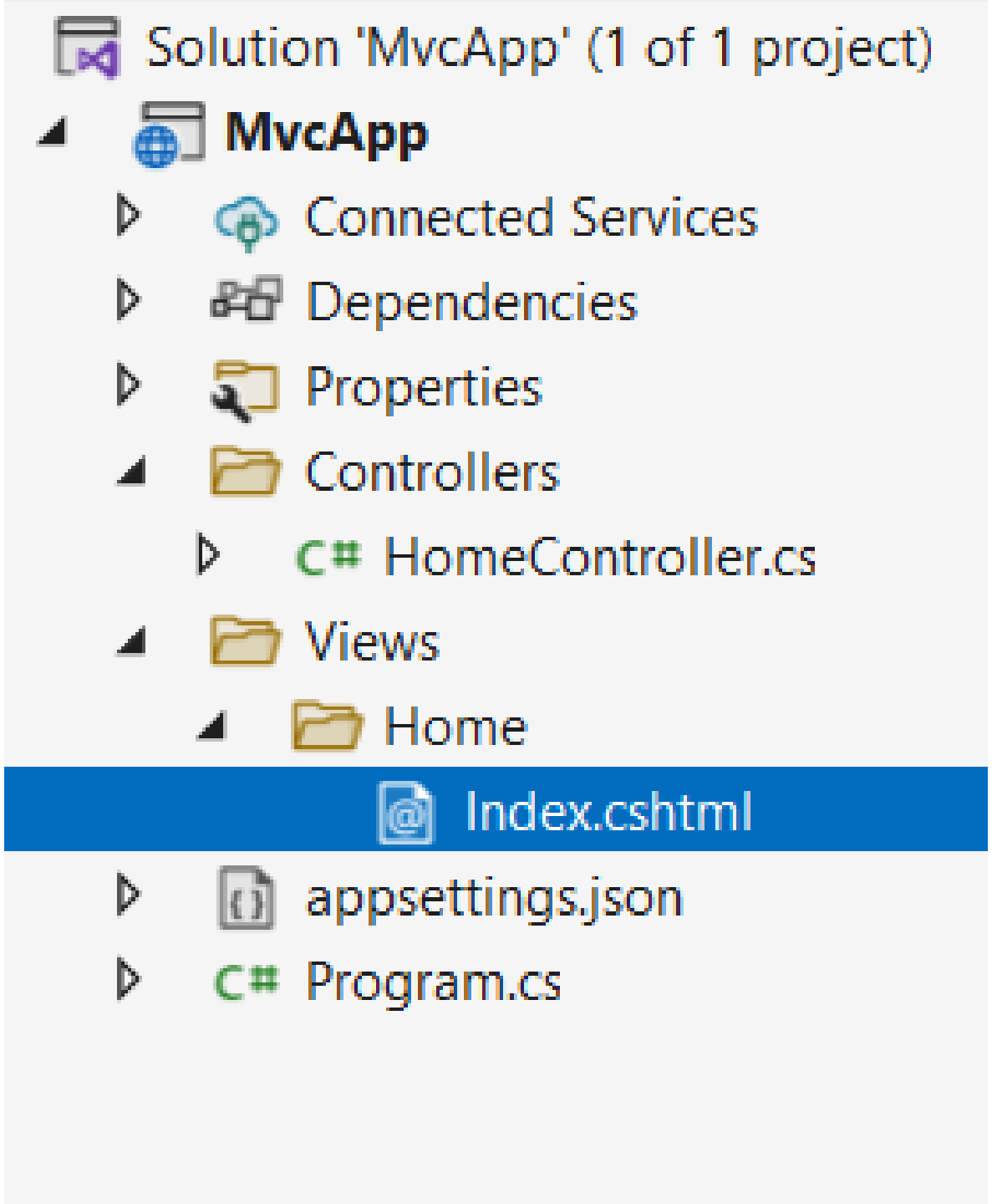
Type: Visual C#

Razor View Page

Name: Index.cshtml

Add

Cancel


Solution 'MvcApp' (1 of 1 project)
MvcApp
Connected Services
Dependencies
Properties
Controllers
HomeController.cs
Views
Home
Index.cshtml
appsettings.json
Program.cs

Откроем представление **Index.cshtml**, добавим следующий код:

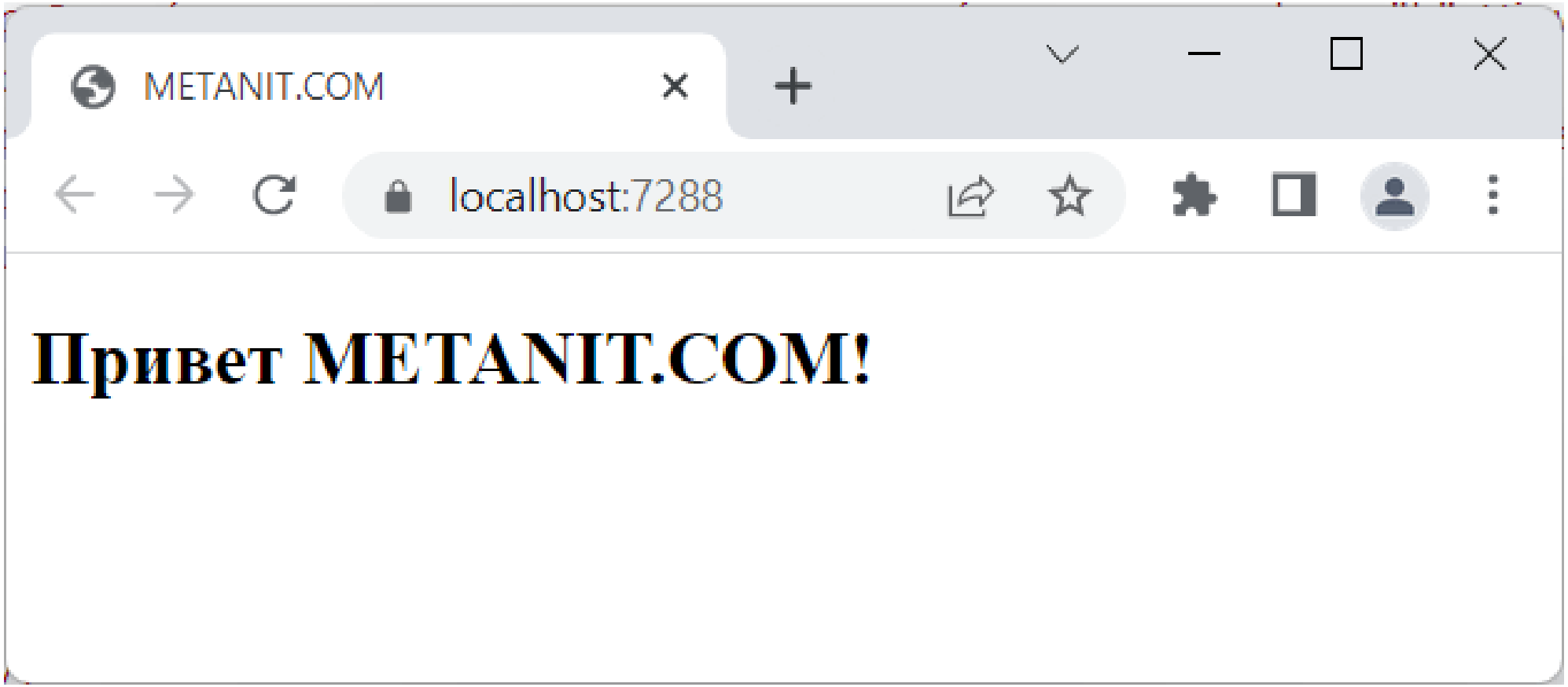
```
<!DOCTYPE html>
<html>
<head>
    <title>METANIT.COM</title>
    <meta charset="utf-8" />
</head>
<body>
    <h2>Привет METANIT.COM!</h2>
</body>
</html>
```

Подключение функционала представлений

Чтобы приложение могло использовать представления, нам надо подключить соответствующие сервисы. Для этого изменим код файла **Program.cs** следующим образом:

```
var builder = WebApplication.CreateBuilder(args);

// добавляем поддержку контроллеров с представлениями
builder.Services.AddControllersWithViews();
var app = builder.Build();

// устанавливаем сопоставление маршрутов с контроллерами
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

И после запуска приложения и обращении к методу Index контроллера Home браузер отобразит нам веб-страницу, которая будет сгенерирована на основе представления **Index.cshtml**.



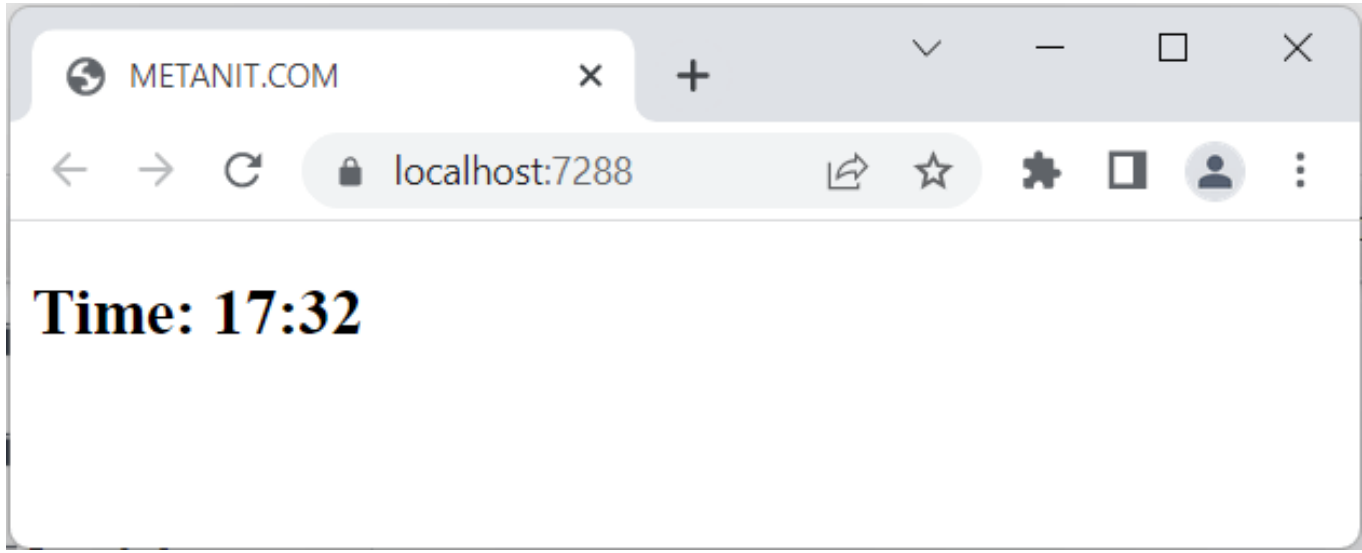
Движок представлений Razor

Представления в ASP.NET Core MVC может содержать не только стандартный код html, но и также вставки кода на языке C#. Для обработки кода, который содержит как элементы html, так и конструкции языка C#, применяется движок представлений.

В действительности при вызове метода View контроллер не производит рендеринг представления и не генерирует разметку html. Контроллер только готовит данные и выбирает, какое представление надо возвратить в качестве объекта ViewResult. Затем уже объект ViewResult обращается к движку представления для рендеринга представления в выходной ответ.

Синтаксис Razor довольно прост - все его конструкции предваряются символом @, после которого происходит переход к коду C#. Например, определим следующее представление:

```
<!DOCTYPE html>
<html>
<head>
    <title>METANIT.COM</title>
    <meta charset="utf-8" />
</head>
<body>
    <h2>Time: @DateTime.Now.ToShortTimeString()</h2>
</body>
</html>
```



Передача данных в представление

Существуют различные способы передачи данных из контроллера в представление:

- ViewData
- ViewBag
- Модель представления

Модель представления

Модель представления является во многих случаях более предпочтительным способом для передачи данных в представление. Для передачи данных в представление используется одна из версий метода View:

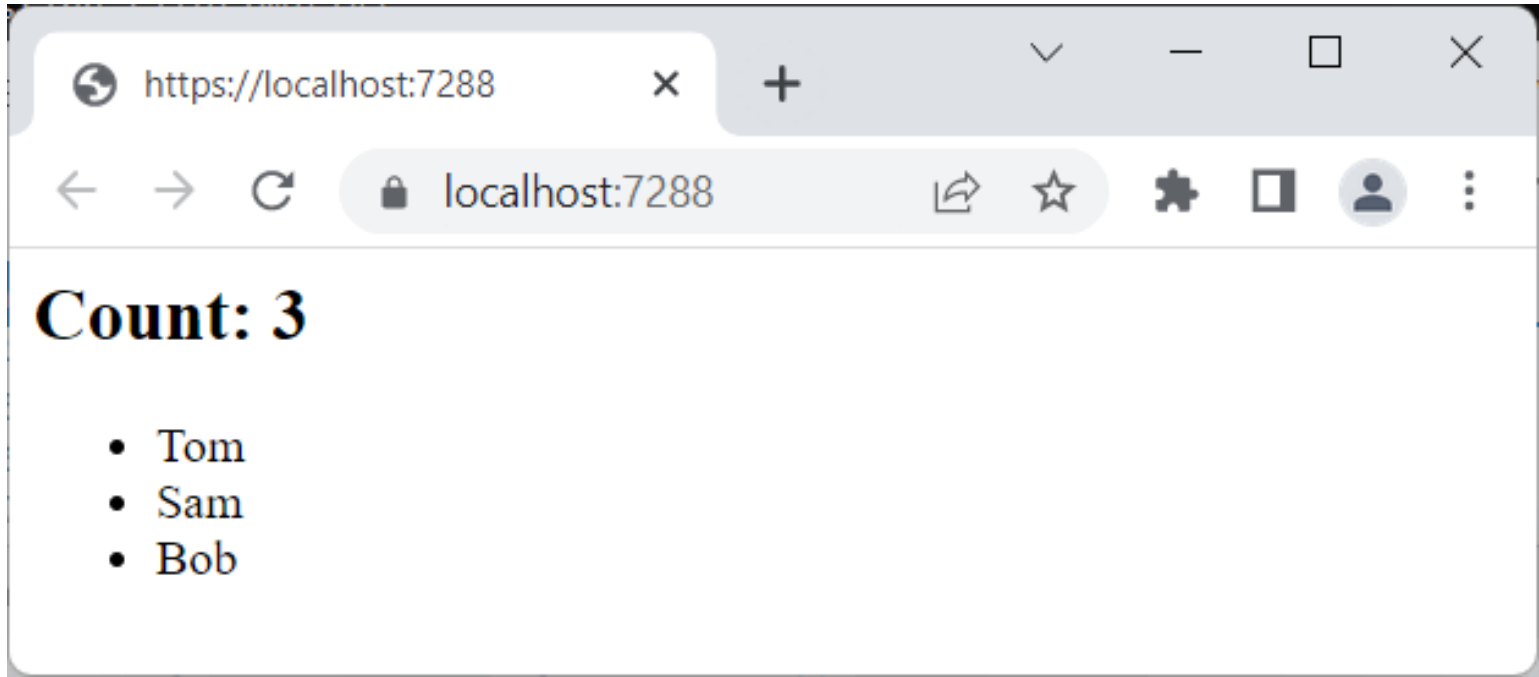
```
public IActionResult Index()
{
    var people = new List<string> { "Tom", "Sam", "Bob" };
    return View(people);
}
```

В метод **View()** передается список, поэтому моделью представления **Index.cshtml** будет тип **List<string>** (либо **IEnumerable<string>**). И теперь в представлении мы можем написать так:

```
@model List<string>

<h2>Count: @Model.Count</h2>
<ul>
@foreach(string person in Model)
{
    <li>@person</li>
}
</ul>
```

В самом начале представления с помощью директивы @model устанавливается модель представления. Тип модели должен совпадать с типом объекта, который передается в метод View() в контроллере.

Установка модели указывает, что объект Model теперь будет представлять объект List<string> или список. И мы сможем использовать **Model** в качестве списка.



Работа с формами

Формы html представляют одну из форм передачи наборов данных на сервер. Обычно для создания форм и их элементов в MVC применяются либо html-хелперы, либо tag-хелперы, которые рассматриваются далее. Однако в данном случае мы рассмотрим взаимодействие на примере стандартных тегов html, которые создают элементы формы.

В одной из прошлых тем было рассмотрено, как MVC связывает отправленные значения формы с параметрами метода. Например, у нас есть действие Index:

```
public class HomeController : Controller
{
    [HttpGet]
    public IActionResult Index() => View();
    [HttpPost]
    public string Index(string username) => $"User Name: {username}";
}
```

Одно действие расщеплено на два метода: GET-версию, которая отдает представление с формой ввода, и POST-версию, которая принимает введенные в эту форму данные.

Теперь создадим само представление. Пусть в папке **Views/Home** есть представление **Index.cshtml** со следующим кодом:

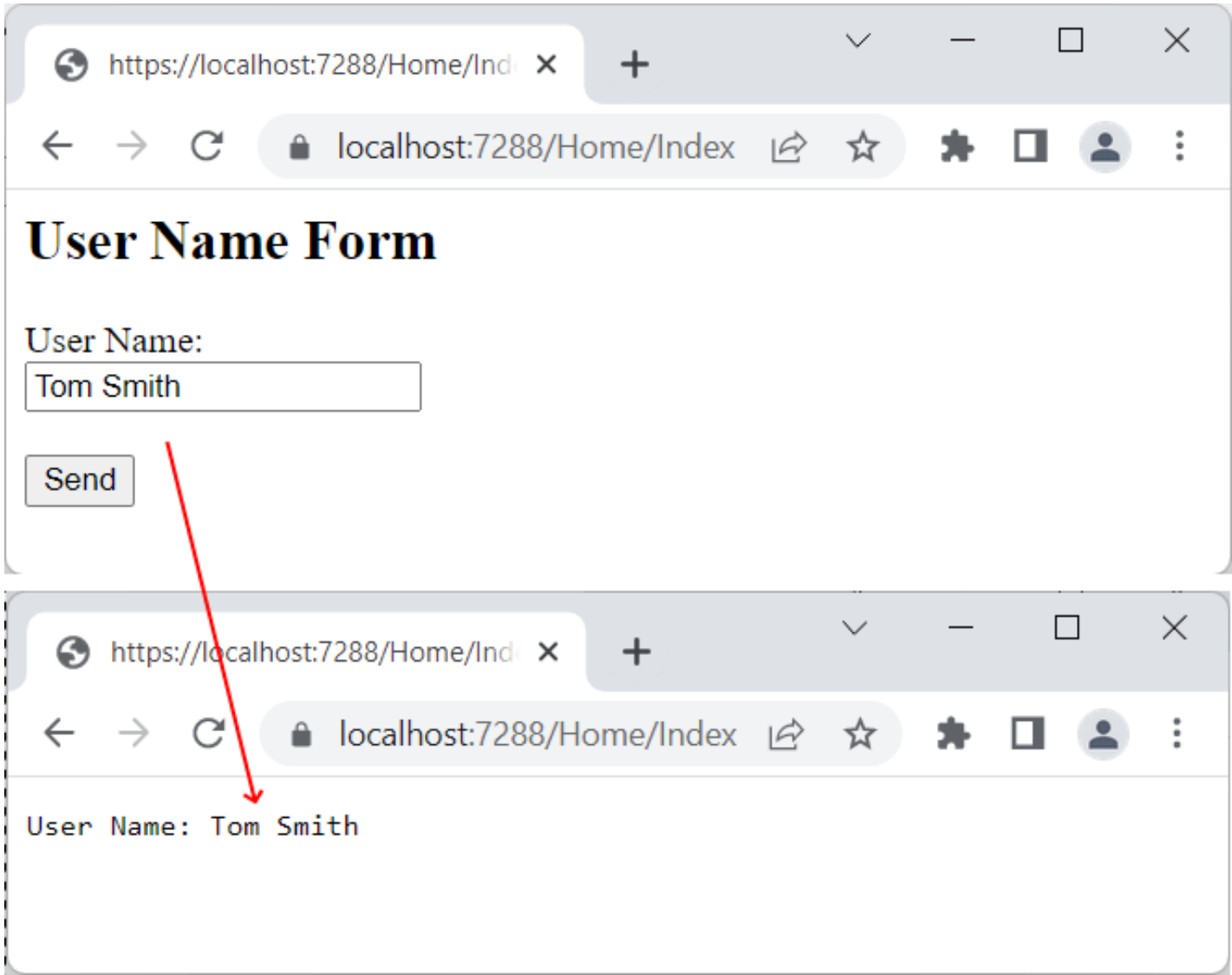
```
<h2>User Name Form</h2>
<form method="post">
    <label>User Name:</label><br />
    <input type="text" name="username" /><br />
    <input type="submit" value="Send" />
</form>
```

Чтобы инфраструктура MVC могла автоматически связать данные из формы с параметрами метода, значения атрибутов name у полей формы совпадают с именами параметров.

```
<input type="text" name="username" /><br />
```

```
public string Index(string username) => $"User Name: {username}";
```

Таким образом, когда форма отправится на сервер, при обработке запроса фреймворк MVC автоматически свяжет значения полей формы с параметрами:



Модели

Одним из ключевых компонентов паттерна MVC являются модели. Ключевая задача моделей - описание структуры и логики используемых данных.

Как правило, все используемые сущности в приложении выделяются в отдельные модели, которые и описывают структуру каждой сущности. В зависимости от задач и предметной области мы можем выделить различное количество моделей в приложении.

Все модели оформляются как обычные **POCO-классы** (plain-old CRL objects), то есть обычные классы на языке C#. Например, если мы работаем с данными пользователей, то мы могли бы определить в проекте следующую модель, которая представляет пользователя:

```
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public int Age { get; set; }
}
```

Модель Person определяет ряд свойств: уникальный идентификатор Id, имя и возраст пользователя. Это классическая **анемичная модель**. **Анемичная** модель не имеет поведения и хранит только состояние в виде свойств.

В приложении **ASP.NET MVC Core** модели можно разделить по степени применения на несколько групп:

- Модели, объекты которых хранятся в специальных хранилищах данных (например, в базах данных, файлах xml и т.д.)
- Модели, которые используются для передачи данных в представление или наоборот, для получения данных из представления. Такие модели еще называются моделями представления
- Вспомогательные модели для промежуточных вычислений

Например, создадим новый проект ASP.NET Core по типу **ASP.NET Core Empty**. Добавим в него папку **Models**, в которую добавим новый класс Person

```
namespace MvcApp.Models
{
    public record class Person(int Id, string Name, int Age);
}
```

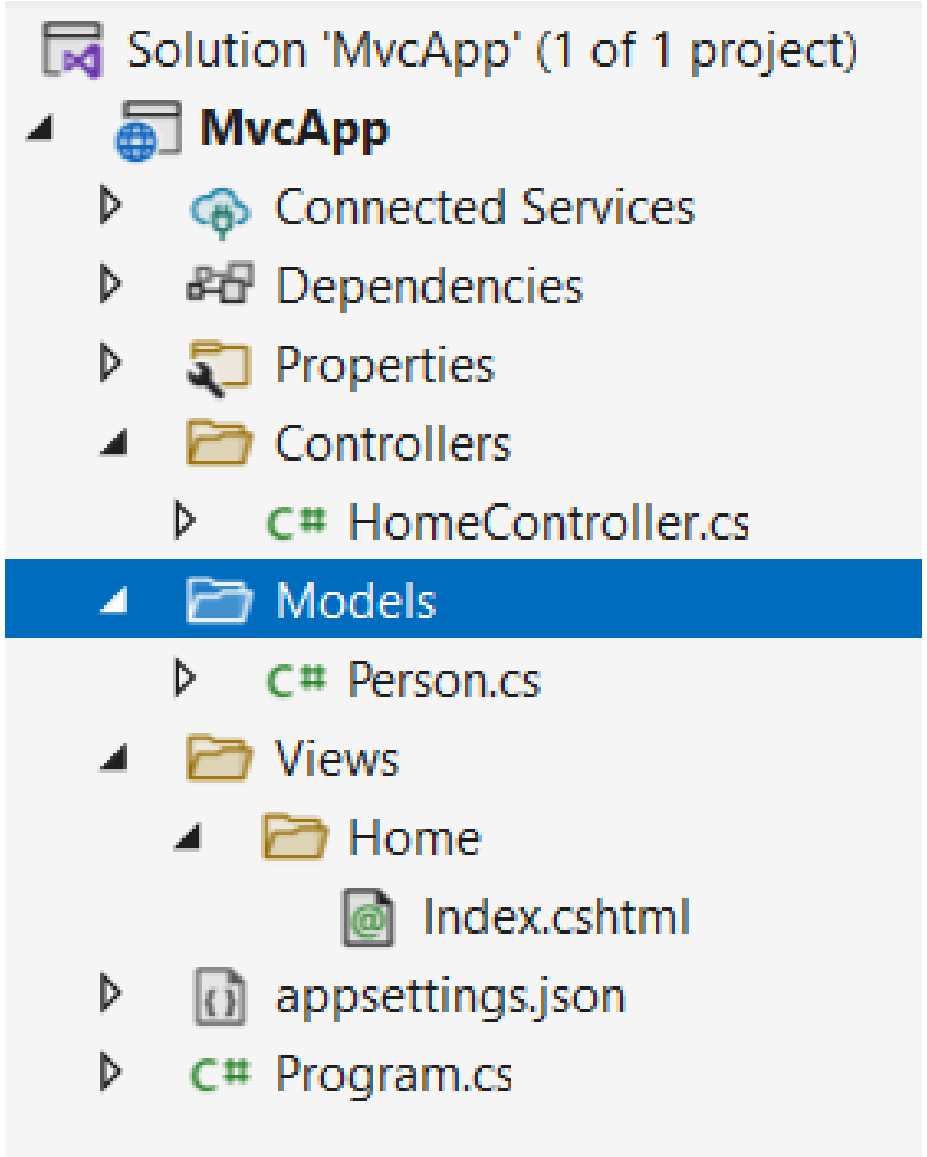
Далее добавим в проект папку **Controllers**, а в нее новый класс - **HomeController** со следующим кодом:

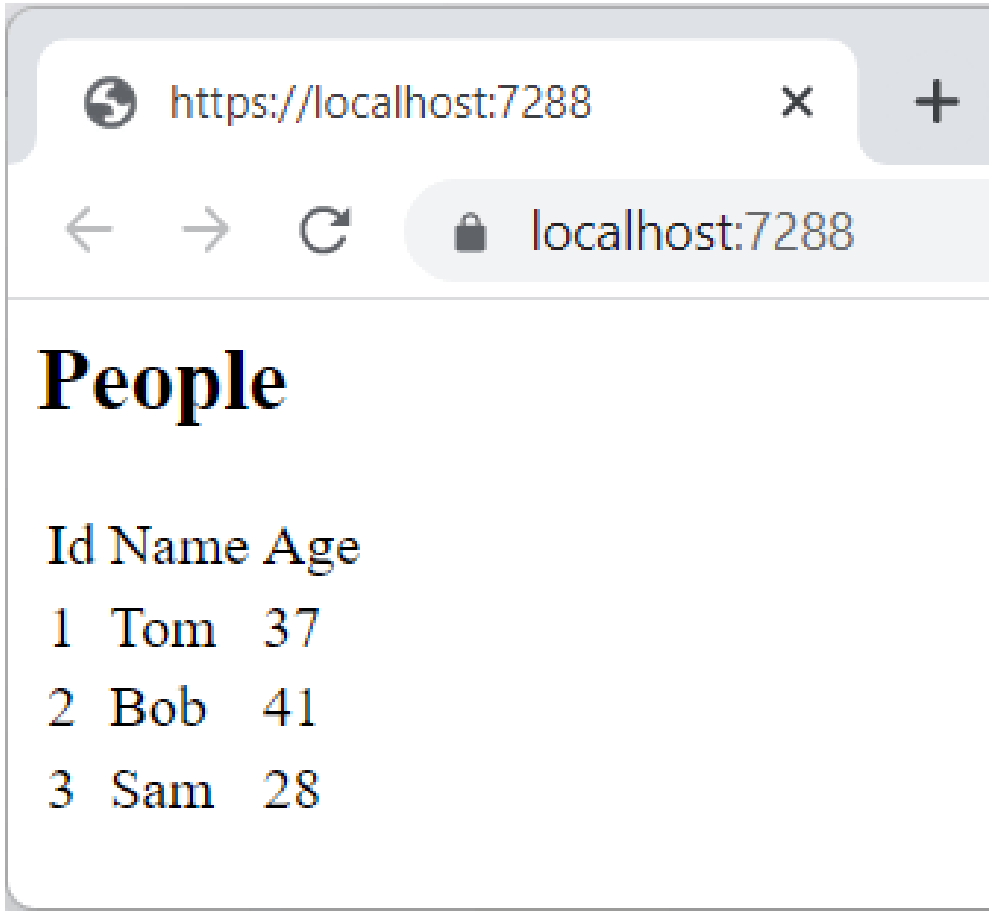
```
public class HomeController : Controller
{
    List<Person> people = new List<Person>
        {
            new Person(1, "Tom", 37),
            new Person(2, "Bob", 41),
            new Person(3, "Sam", 28)
        };
    public IActionResult Index()
    {
        return View(people);
    }
}
```

И в конце добавим в проект папку **Views**, а в нее - каталог **Home**. Далее в папку **Views/Home** добавим представление **Index.cshtml**, которое будет выводить все объекты:

```
@using MvcApp.Models
@model IEnumerable<Person>

<h2>People</h2>
<table class="table">
        <tr><td>Id</td><td>Name</td><td>Age</td></tr>
    @foreach (var p in Model)
    {
        <tr><td>@p.Id</td><td>@p.Name</td><td>@p.Age</td></tr>
    }
</table>
```

Структура проекта и результат:





Tag-хелперы

Tag-хелперы представляют собой функциональность, предназначенную для генерации HTML-разметки. Tag-хелперы применяются в представлениях и выглядят как обычные html-элементы или атрибуты, однако при работе приложения они обрабатываются движком Razor на стороне сервера и в конечном счете преобразуются в стандартные html-элементы.

Для примера в проекте определим папку **Controllers**, а в нее поместим следующий контроллер **HomeController**:

```
public class HomeController : Controller
{
    public IActionResult Index() => View();
    public string Contacts() => "Contacts page";
}
```

Для представлений этого контроллера создадим в проекте папку **Views**, а в ней - каталог **Home**. Затем в папку **Views/Home** поместим новое представление **Index.cshtml**:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

<a asp-controller="Home" asp-action="Contacts">Контакты</a>
```

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Первый параметр директивы указывает на tag-хелперы, которые будут доступны в представлении, а второй параметр определяет библиотеку хелперов. В данном случае директива использует синтаксис подстановок - знак звездочки ("*") означает, что подключаются все хелперы из библиотеки **Microsoft.AspNetCore.Mvc.TagHelpers**.

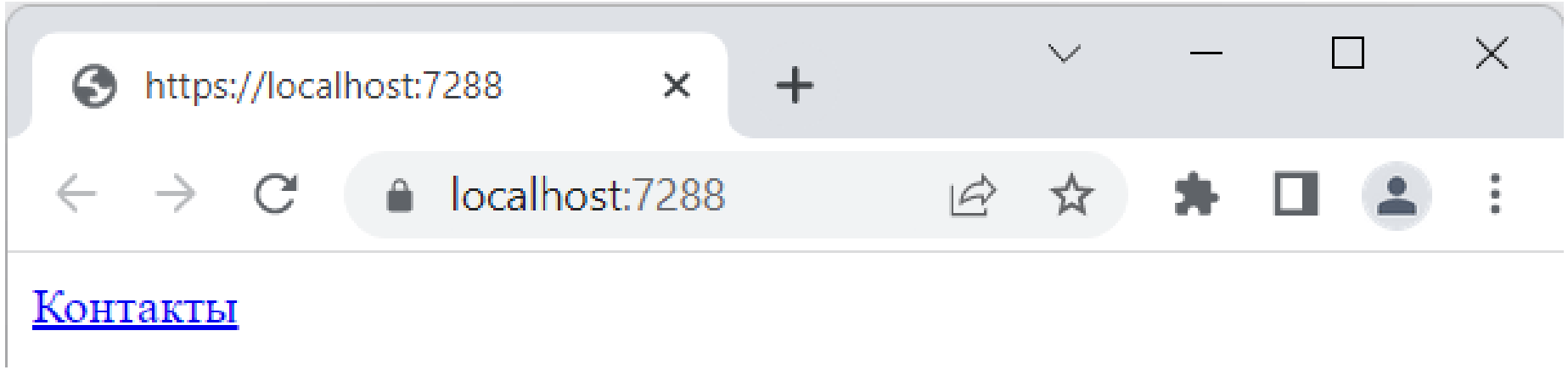
```
<a asp-controller="Home" asp-action="Contacts">Контакты</a>
```

Внешне данный хелпер напоминает обычную ссылку - стандартный элемент html, однако это не элемент html.

```
<a asp-controller="Home" asp-action="Contacts">Контакты</a>
```

Данный хелпер создает ссылку, для которой в качестве контроллера используется Home, а в качестве метода Contact.

В итоге при запуске проекта вместо данного tag-хелпера будет сформирована гиперссылка, по нажатию на которую запрос будет обрабатываться методом Contacts контроллера Home:



_ViewImports.cshtml и @addTagHelper

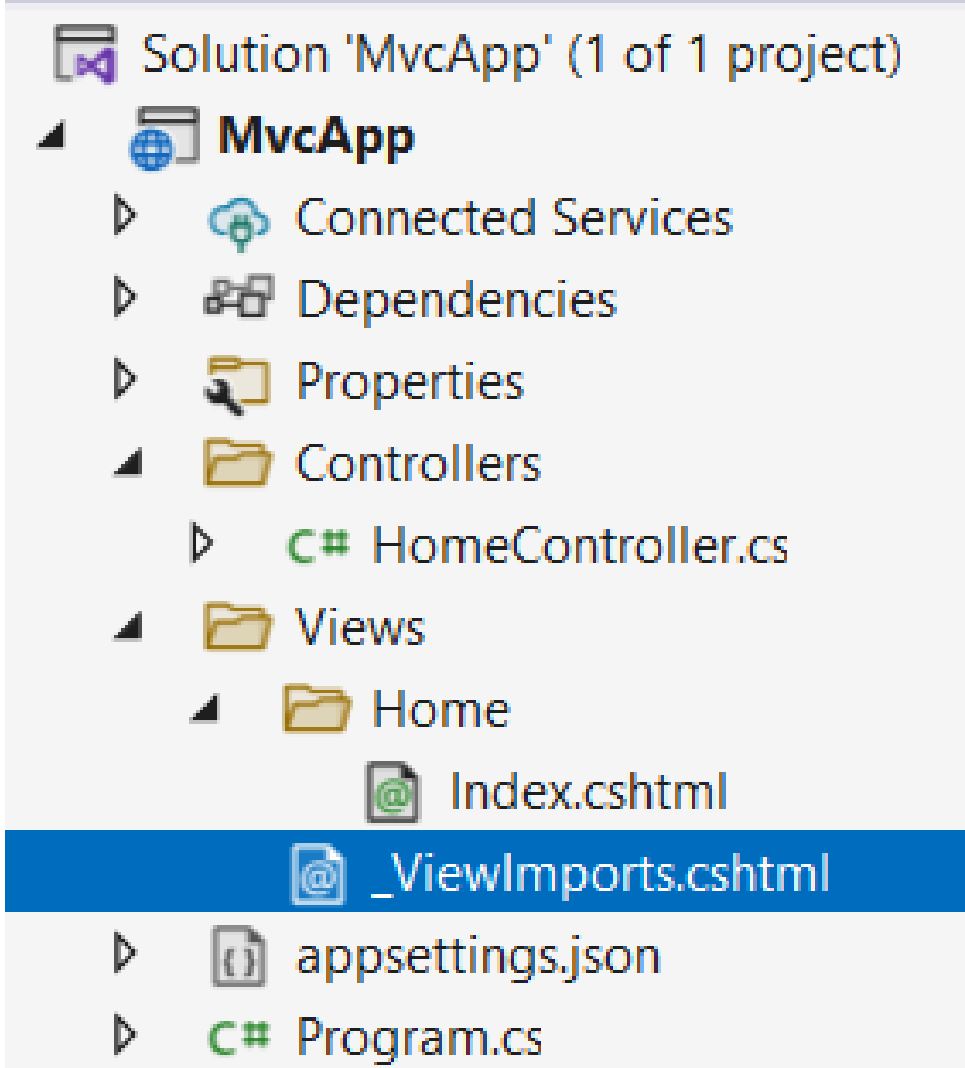
Выше в представление были подключены tag-хелперы. Но что, если нам надо подключить tag-хелперы в кучу представлений? Вместо того, чтобы прописывать директиву **@addTagHelper** в каждом отдельном представлении, мы можем подключить все хелперы разом. Для этого применяется файл **_ViewImports.cshtml**.

Итак, добавим в проект в папку **Views** новый файл **_ViewImports.cshtml**:

В файле **_ViewImports.cshtml** определим подключение tag-хелперов:

```
@addTagHelper *,
Microsoft.AspNetCore.Mvc.TagHelpers
```

После этого из представления Index.cshtml можно удалить подключение tag-хелперов и оставить только создание ссылки



Tag-хелперы форм

Рассмотрим применение хелперов на примере следующих моделей:

```
public record class Product(string Name, int Price, int CompanyId);

public record class Company(int Id, string Name);
```

И допустим в контроллере определено действие Create для создания нового объекта Product:

```
public class HomeController : Controller
{
    IEnumerable<Company> companies = new List<Company>
        {
            new Company( 1, "Apple"),
            new Company(2, "Samsung"),
            new Company(3, "Google")
        };

    public IActionResult Create()
    {
        ViewBag.Companies = new SelectList(companies,
                                                "Id", "Name");
        return View();
    }
```

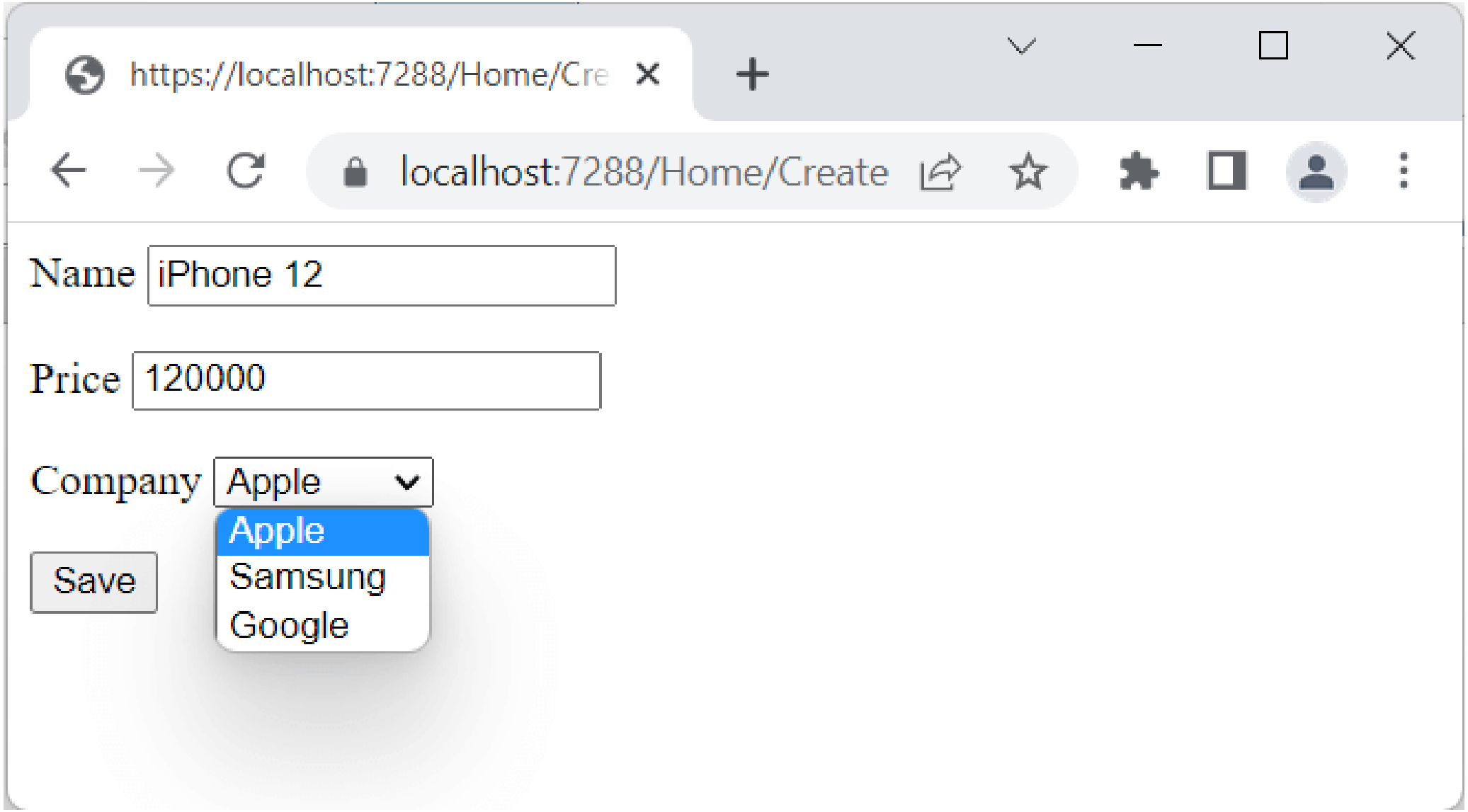
Запишем обработчик post запроса:

```
[HttpPost]
public string Create(Product product)
{
    Company company = companies.FirstOrDefault(c =>
                                    c.Id == product.CompanyId);

    return $"Добавлен новый элемент: {product.Name}
                                        ({company?.Name})";
}
```

И допустим в контроллере определено действие Create для создания нового объекта Product:

```
@model MvcApp.Models.Product

<h2>Добавление телефона</h2>

<form asp-action="Create"
      asp-controller="Home"
      asp-antiforgery="true">

На след. слайде

</form>
```

```
<div>
    <p>
        <label asp-for="Name"></label>
        <input type="text" asp-for="Name" />
    </p>
    <p>
        <label asp-for="Price"></label>
        <input asp-for="Price" />
    </p>
    <p>
        <label asp-for="CompanyId">Company</label>
        <select asp-for="CompanyId"
                asp-items="ViewBag.Companies"></select>
    </p>
    <p>
        <input type="submit" value="Save" />
    </p>
</div>
```


https://localhost:7288/Home/Cre
localhost:7288/Home/Create
Name
iPhone 12
Price
120000
Company
Apple
Apple
Samsung
Google
Save